

Instagram Automation Platform (2026)

Overview

This project is a production-style Instagram automation framework built with Python using:

- `instagrapi` as the primary Instagram client
- Docker support
- Multi-account orchestration
- Smart randomized delays
- Config-driven automation
- Optional Playwright browser layer for safer login/session verification
- Structured logging
- Proxy support
- Queue-based task execution
- 2FA login support
- Media posting support (photo, carousel, reels, stories)
- Scraping utilities

The architecture is modular and intended for educational, testing, and internal automation workflows.

Tech Stack

- Python 3.12+
 - `instagrapi`
 - Playwright
 - Redis (optional)
 - Docker + Docker Compose
 - APScheduler
 - Faker
 - Rich logging
 - Pydantic
-

Recommended Base Repository (2026)

Primary Base:

`instagrapi`

Official Repo:

<https://github.com/subzeroid/instagrapi>

Why:

- Most stable maintained private API wrapper
 - Reliable media upload support
 - Session persistence
 - 2FA support
 - Story/Reel support
 - Better than legacy instagram-private-api
-

Full Project Structure

```
instagram-automation/
├── app/
│   ├── core/
│   │   ├── client_manager.py
│   │   ├── session_manager.py
│   │   ├── proxy_manager.py
│   │   ├── randomizer.py
│   │   ├── scheduler.py
│   │   ├── logger.py
│   │   ├── security.py
│   │   └── fingerprint.py
│   ├── actions/
│   │   ├── post_photo.py
│   │   ├── post_reel.py
│   │   ├── post_story.py
│   │   ├── post_carousel.py
│   │   ├── auto_like.py
│   │   ├── auto_comment.py
│   │   ├── auto_follow.py
│   │   ├── auto_unfollow.py
│   │   └── engagement.py
│   ├── scraping/
│   │   ├── scrape_followers.py
│   │   ├── scrape_hashtags.py
│   │   ├── scrape_locations.py
│   │   └── scrape_competitors.py
│   ├── browser/
│   │   ├── playwright_login.py
│   │   └── human_behavior.py
│   ├── config/
│   │   └── settings.py
```

```
| | | └─ config.json
| | |
| | └─ utils/
| |   └─ helpers.py
| |   └─ captions.py
| |   └─ media_validator.py
| |
| └─ main.py
|
└─ data/
  └─ sessions/
  └─ logs/
  └─ exports/
  └─ media/
|
└─ docker/
  └─ Dockerfile
  └─ docker-compose.yml
|
└─ requirements.txt
└─ .env.example
└─ README.md
└─ run.py
```

requirements.txt

```
instagrapi==2.1.5
playwright==1.52.0
faker==37.1.0
python-dotenv==1.1.0
rich==14.0.0
pydantic==2.11.4
APScheduler==3.11.0
requests==2.32.3
PySocks==1.7.1
redis==6.0.0
```

.env.example

```
IG_USERNAME=username
IG_PASSWORD=password
```

```
IG_PROXY=socks5://127.0.0.1:9050  
HEADLESS=true
```

config.json

```
{  
  "accounts": [  
    {  
      "username": "account1",  
      "password": "password1",  
      "proxy": "socks5://127.0.0.1:9050"  
    }  
  ],  
  "limits": {  
    "likes_per_day": 120,  
    "comments_per_day": 40,  
    "follows_per_day": 40,  
    "unfollows_per_day": 40  
  },  
  "delays": {  
    "min_seconds": 12,  
    "max_seconds": 45  
  },  
  "hashtags": [  
    "ai",  
    "startup",  
    "tech"  
  ],  
  "competitors": [  
    "example_account"  
  ]  
}
```

app/config/settings.py

```
from dotenv import load_dotenv  
import os  
import json  
  
load_dotenv()  
  
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))  
CONFIG_PATH = os.path.join(BASE_DIR, "config", "config.json")
```

```
with open(CONFIG_PATH, "r") as f:
    CONFIG = json.load(f)

ENV = {
    "IG_USERNAME": os.getenv("IG_USERNAME"),
    "IG_PASSWORD": os.getenv("IG_PASSWORD"),
    "HEADLESS": os.getenv("HEADLESS", "true")
}
```

app/core/logger.py

```
from rich.console import Console
from rich.logging import RichHandler
import logging

console = Console()

logging.basicConfig(
    level="INFO",
    format="%(message)s",
    handlers=[RichHandler()]
)

logger = logging.getLogger("instagram-bot")
```

app/core/randomizer.py

```
import random
import time

def smart_sleep(min_seconds=10, max_seconds=30):
    delay = random.uniform(min_seconds, max_seconds)
    time.sleep(delay)
```

app/core/proxy_manager.py

```
class ProxyManager:
    def __init__(self, proxies: list[str]):
        self.proxies = proxies
```

```
        self.index = 0

    def get_proxy(self):
        proxy = self.proxies[self.index]
        self.index = (self.index + 1) % len(self.proxies)
        return proxy
```

app/core/client_manager.py

```
from instagrapi import Client
from app.core.logger import logger

class InstagramClientManager:
    def __init__(self, username, password, proxy=None):
        self.username = username
        self.password = password
        self.proxy = proxy
        self.client = Client()

    def login(self):
        if self.proxy:
            self.client.set_proxy(self.proxy)

        self.client.delay_range = [1, 3]

        try:
            self.client.login(self.username, self.password)
            logger.info(f"Logged in: {self.username}")

        except Exception as e:
            logger.error(str(e))
            raise

        return self.client
```

app/core/session_manager.py

```
import os

class SessionManager:
    SESSION_DIR = "data/sessions"
```

```
@classmethod
def session_path(cls, username):
    return os.path.join(cls.SESSION_DIR, f"{username}.json")
```

app/core/fingerprint.py

```
from faker import Faker

fake = Faker()

def random_device_profile():
    return {
        "user_agent": fake.user_agent(),
        "timezone": fake.timezone(),
        "locale": fake.locale()
    }
```

app/core/security.py

```
from app.core.logger import logger

class SafetyManager:
    @staticmethod
    def validate_limits(current, maximum):
        if current >= maximum:
            logger.warning("Daily action limit reached")
            return False
        return True
```

app/actions/post_photo.py

```
from pathlib import Path

def post_photo(client, image_path, caption=""):
    image_path = Path(image_path)

    if not image_path.exists():
```

```
        raise FileNotFoundError(image_path)

    return client.photo_upload(
        path=str(image_path),
        caption=caption
    )
```

app/actions/post_reel.py

```
from pathlib import Path

def post_reel(client, video_path, caption=""):
    video_path = Path(video_path)

    return client.clip_upload(
        path=str(video_path),
        caption=caption
    )
```

app/actions/post_story.py

```
from pathlib import Path

def post_story(client, media_path):
    media_path = Path(media_path)

    if media_path.suffix.lower() in [".jpg", ".jpeg", ".png"]:
        return client.photo_upload_to_story(str(media_path))

    return client.video_upload_to_story(str(media_path))
```

app/actions/post_carousel.py

```
from pathlib import Path

def post_carousel(client, media_paths, caption=""):
    media = [Path(p) for p in media_paths]
```

```
    return client.album_upload(  
        paths=[str(m) for m in media],  
        caption=caption  
    )
```

app/actions/auto_like.py

```
from app.core.randomizer import smart_sleep  
  
def like_media(client, media_ids):  
    for media_id in media_ids:  
        client.media_like(media_id)  
        smart_sleep()
```

app/actions/auto_comment.py

```
import random  
from app.core.randomizer import smart_sleep  
  
COMMENTS = [  
    "Amazing content 🔥",  
    "Love this 🙌",  
    "Great post 🚀",  
    "Very inspiring ✨"  
]  
  
def comment_media(client, media_ids):  
    for media_id in media_ids:  
        client.media_comment(media_id, random.choice(COMMENTS))  
        smart_sleep()
```

app/actions/auto_follow.py

```
from app.core.randomizer import smart_sleep  
  
def follow_users(client, user_ids):
```

```
for user_id in user_ids:
    client.user_follow(user_id)
    smart_sleep()
```

app/actions/auto_unfollow.py

```
from app.core.randomizer import smart_sleep

def unfollow_users(client, user_ids):
    for user_id in user_ids:
        client.user_unfollow(user_id)
        smart_sleep()
```

app/actions/engagement.py

```
from app.actions.auto_like import like_media
from app.actions.auto_comment import comment_media

def engage(client, media_ids):
    like_media(client, media_ids)
    comment_media(client, media_ids)
```

app/scraping/scrape_followers.py

```
import json

def scrape_followers(client, username):
    user_id = client.user_id_from_username(username)
    followers = client.user_followers(user_id)

    data = [
        {
            "username": u.username,
            "full_name": u.full_name
        }
        for u in followers.values()
    ]
```

```
with open(f"data/exports/{username}_followers.json", "w") as f:
    json.dump(data, f, indent=2)
```

app/scraping/scrape_hashtags.py

```
import json

def scrape_hashtag(client, hashtag):
    medias = client.hashtag_medias_recent(hashtag, amount=20)

    data = [
        {
            "id": media.id,
            "caption": media.caption_text
        }
        for media in medias
    ]

    with open(f"data/exports/{hashtag}.json", "w") as f:
        json.dump(data, f, indent=2)
```

app/scraping/scrape_locations.py

```
import json

def scrape_location(client, location_id):
    medias = client.location_medias_recent(location_id, amount=20)

    with open(f"data/exports/location_{location_id}.json", "w") as f:
        json.dump([m.dict() for m in medias], f, indent=2)
```

app/scraping/scrape_competitors.py

```
from app.scraping.scrape_followers import scrape_followers

def scrape_competitors(client, usernames):
```

```
for username in usernames:
    scrape_followers(client, username)
```

app/browser/playwright_login.py

```
from playwright.sync_api import sync_playwright
import time
import random

class BrowserLogin:
    def __init__(self, headless=True):
        self.headless = headless

    def login(self, username, password):
        with sync_playwright() as p:
            browser = p.chromium.launch(headless=self.headless)
            page = browser.new_page()

            page.goto("https://www.instagram.com/accounts/login/")

            time.sleep(random.uniform(3, 6))

            page.fill('input[name="username"]', username)
            time.sleep(random.uniform(1, 2))

            page.fill('input[name="password"]', password)
            time.sleep(random.uniform(1, 2))

            page.click('button[type="submit"]')

            time.sleep(10)

            browser.close()
```

app/browser/human_behavior.py

```
import random
import time

def typing_delay():
    time.sleep(random.uniform(0.05, 0.15))
```

```
def random_pause():
    time.sleep(random.uniform(1, 5))
```

app/utils/media_validator.py

```
from pathlib import Path

SUPPORTED = [
    ".jpg",
    ".jpeg",
    ".png",
    ".mp4"
]

def validate_media(path):
    path = Path(path)

    if path.suffix.lower() not in SUPPORTED:
        raise ValueError("Unsupported media format")
```

app/utils/captions.py

```
CAPTIONS = [
    "🚀 Growth mode activated",
    "✨ Consistency wins",
    "🔥 Building every day"
]
```

app/main.py

```
from app.config.settings import CONFIG
from app.core.client_manager import InstagramClientManager
from app.actions.post_photo import post_photo

def main():
    for account in CONFIG["accounts"]:
```

```
manager = InstagramClientManager(
    username=account["username"],
    password=account["password"],
    proxy=account.get("proxy")
)

client = manager.login()

print(f"Running automation for {account['username']}")

# Example upload
# post_photo(client, "data/media/test.jpg", "Hello from automation")

if __name__ == "__main__":
    main()
```

run.py

```
from app.main import main

if __name__ == "__main__":
    main()
```

docker/Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "run.py"]
```

docker/docker-compose.yml

```
version: '3.9'

services:
  instagram-bot:
    build:
      context: ..
      dockerfile: docker/Dockerfile

    volumes:
      - ../:/app

    env_file:
      - ../.env
```

README.md

Instagram Automation Platform (2026)

Production-ready Instagram automation framework built with Python, instagrapi, Playwright, Docker, proxy rotation, smart delays, and multi-account orchestration.

Features

- Multi-account support
- 2FA login support
- Photo, Reel, Story, and Carousel uploads
- Auto-like, auto-comment, follow/unfollow
- Scraping followers, hashtags, locations, competitors
- Smart delays and randomized behavior
- Proxy rotation support
- Docker deployment
- JSON + ENV configuration
- Rich logging
- APScheduler-ready architecture

Installation

```
git clone https://github.com/YOUR_USERNAME/instagram-automation.git
cd instagram-automation
python -m venv venv
source venv/bin/activate
```

```
pip install -r requirements.txt
playwright install
```

Run

```
python run.py
```

Docker

```
docker compose -f docker/docker-compose.yml build
docker compose -f docker/docker-compose.yml up
```

Safe Limits (2026)

Action	Suggested Daily Limit
Likes	80-120
Comments	20-40
Follows	20-40
Unfollows	20-40

Anti-Detection

- Randomized delays
- Session persistence
- Device fingerprint rotation
- Humanized interaction timing
- Residential proxy support

Recommended Base Repository

<https://github.com/subzeroid/instagrapi>

Disclaimer

Instagram internal APIs change frequently. Use responsibly and comply with applicable platform policies.

License

MIT